

Vision Transformers 커스텀 데이터 Fine-tuning

JD-edu

2025 all right reserved

ViT 커스텀 데이터 Fine-tuning

데이터셋

cans cups pets 3개 라벨 각 100여개의 이미지를 fine-tuning

트레이닝 환경

구글 colab과 conda 환경의 로컬 PC에서 트레이닝

구글 colab

- ViT_fine_tuning_CAN.ipynb

로컬 PC conda 환경

- ViT_fine_tuning_CAN.py

구글 Colab 커스텀 데이터 로딩

01

노트북 업로드

ViT_fine_tuning_CAN.ipynb 파일을 자신의 colab에 업로드

02

데이터 업로드

깃허브에서 받은 images 폴더(하위에 cans, pets, cups)를

같은 계정의 구글 드라이브에 업로드

- 루트 폴더에 업로드 권장

03

드라이브 마운트

다음 코드를 실행하면 colab 환경에서 내 구글 드라이브

/drive폴더에서 images를 access 할 수 있음

첫번째 코드 블록 실행

```
import torch

import numpy as np

from PIL import Image

from transformers import ViTForImageClassification, ViTImageProcessor, TrainingArguments, Trainer

from torchvision.transforms import (Compose, Normalize, RandomHorizontalFlip, RandomResizedCrop, ToTensor, Resize, CenterCrop)

from torch.utils.data import DataLoader
```

두번째 블록 실행

```
from google.colab import drive

drive.mount('/content/drive')
```

세번째 마운트 확인

```
!ls /content/drive/MyDrive/images

cans cups pets
```

- 이후에는 colab의 코드 블록을 계속 따라가기

로컬 PC에서 데이터 로딩



1

트레이닝용 파이썬 스크립트 확인

- ViT_fine_tuning_CAN.py

2

데이터 폴더 카피

깃헙에서 다운받은 images 폴더(can, pet, cup)를 트레이닝용 파이썬 스크립트 폴더에 카피

3

스크립트 실행

conda 가상환경 vit_env에서 트레이닝용 파이썬 스크립트 실행

트레이닝 파이썬 스크립트 해석

필요 파이썬 라이브러리 импорт, 원본 이미지 폴더 지정 및 데이터 로딩

```
import torch

import numpy as np

from PIL import Image

from transformers import ViTForImageClassification, ViTImageProcessor,
TrainingArguments, Trainer

from torchvision.transforms import (Compose, Normalize,
RandomHorizontalFlip, RandomResizedCrop, ToTensor, Resize, CenterCrop)

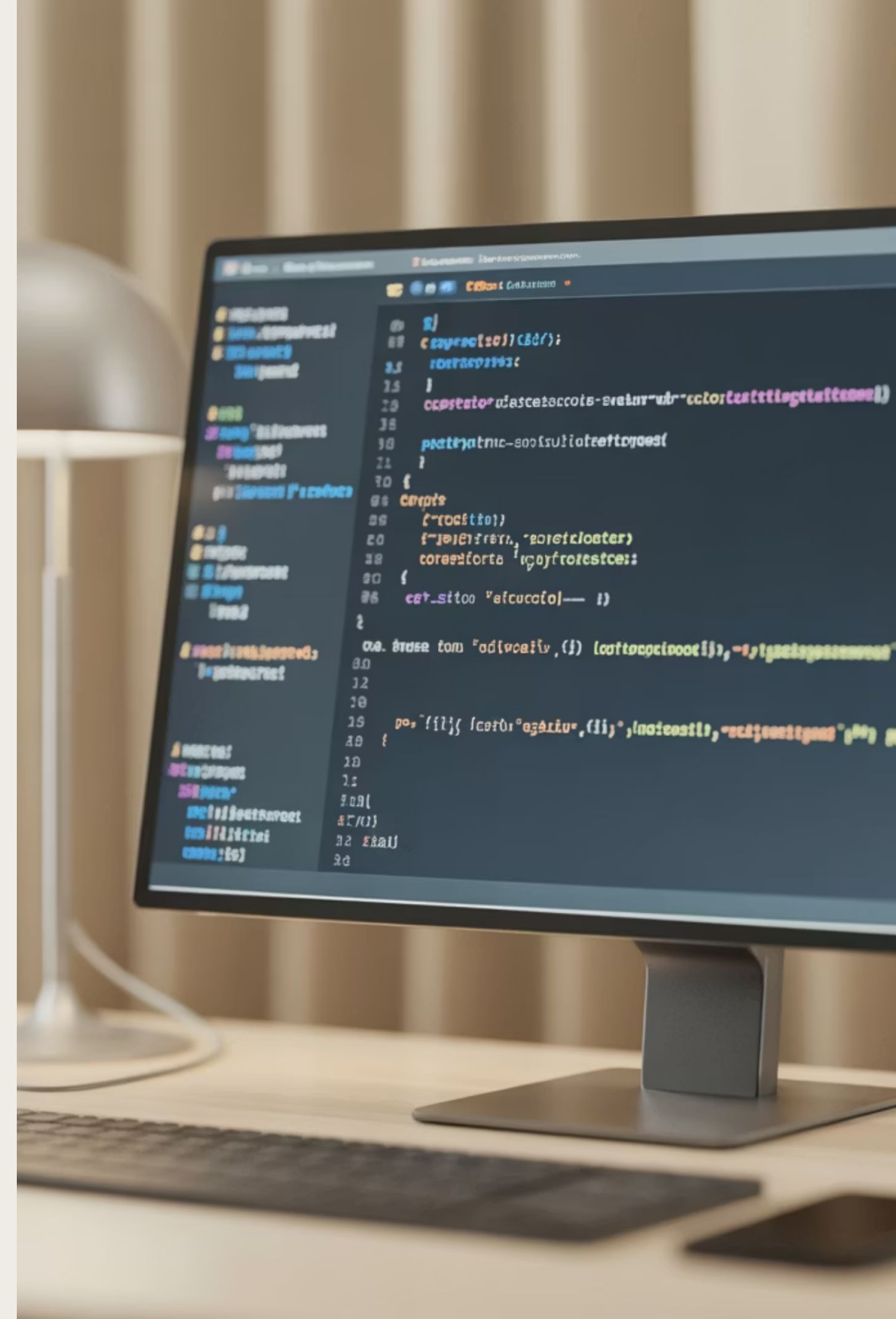
from torch.utils.data import DataLoader
```

```
CUSTOM_DATA_ROOT = "./images"

# 'can', 'pet', 'cup' 폴더가 포함된 상위 폴더

from datasets import load_dataset

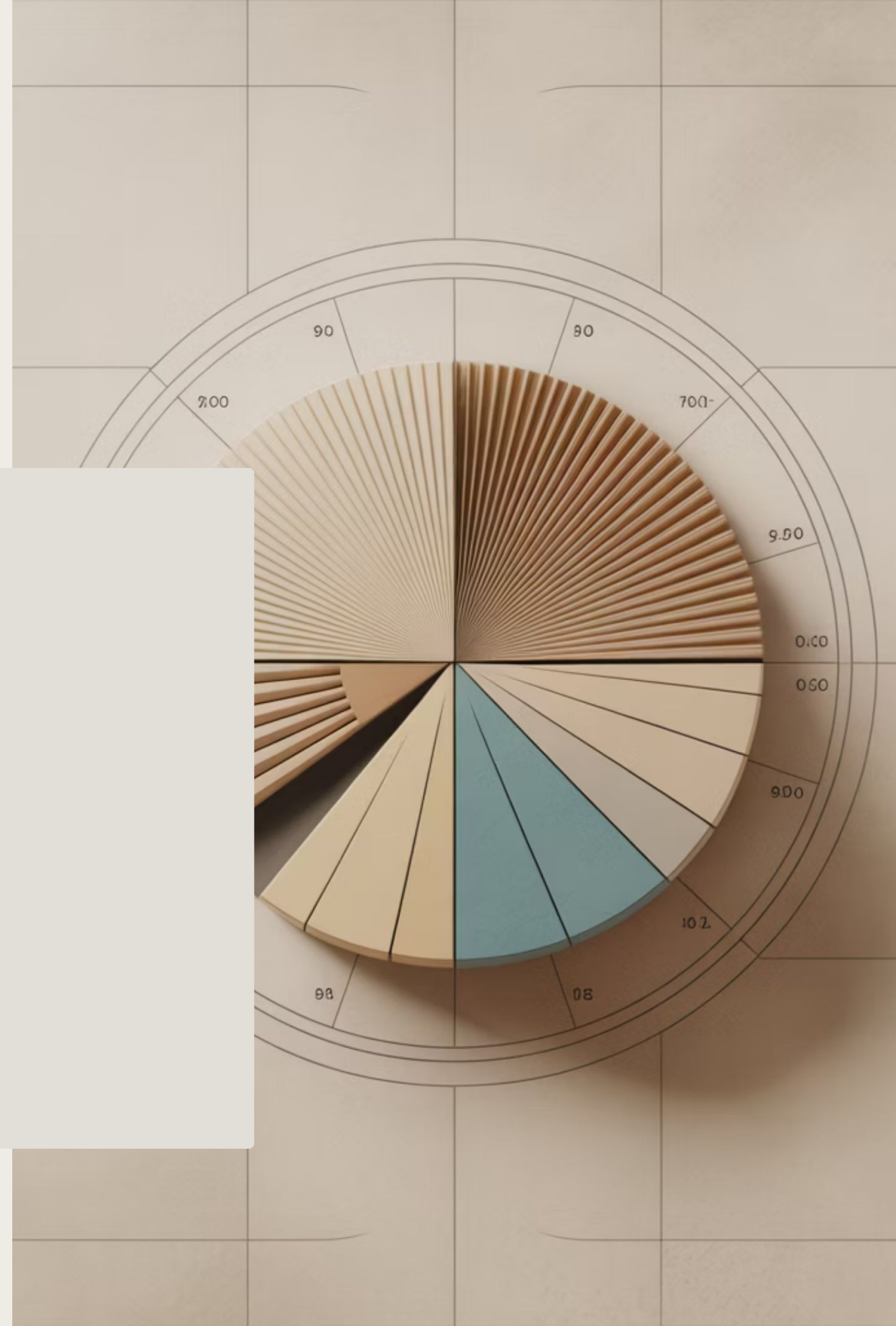
raw_datasets = load_dataset('imagefolder', data_dir=CUSTOM_DATA_ROOT)
```



트레이닝 파이선 스크립트 해석

훈련/검증 데이터 분리

```
splits = raw_datasets['train'].train_test_split(test_size=0.1, seed=42)
train_ds = splits['train']
val_ds = splits['test']
labels = train_ds.features['label'].names
id2label = {i: label for i, label in enumerate(labels)}
label2id = {label: i for i, label in enumerate(labels)}
num_labels = len(labels)
# 3 (can, pet, cup)
print(f"로드된 클래스: {labels}")
print(f"총 훈련 데이터 개수: {len(train_ds)}")
```



트레이닝 파이선 스크립트 해석

베이스 모델(ImageNet) 다운로드 및 전처리 셋팅

```
model_name_or_path = 'google/vit-base-patch16-224-in21k'
processor = ViTImageProcessor.from_pretrained(model_name_or_path)
image_mean, image_std = processor.image_mean, processor.image_std
size = processor.size["height"]
normalize = Normalize(mean=image_mean, std=image_std)
_train_transforms = Compose(
    [RandomResizedCrop(size), RandomHorizontalFlip(), ToTensor(), normalize]
)
_val_transforms = Compose(
    [Resize(size), CenterCrop(size), ToTensor(), normalize]
)
def train_transforms(examples): # 'image' 컬럼은 ImageFolder의 기본 출력 컬럼입니다.
    examples['pixel_values'] = [_train_transforms(image.convert("RGB")) for image in examples['image']]
    return examples
def val_transforms(examples):
    examples['pixel_values'] = [_val_transforms(image.convert("RGB")) for image in examples['image']]
    return examples
```

트레이닝 파이선 스크립트 해석

전처리 적용

```
train_ds.set_transform(train_transforms)
val_ds.set_transform(val_transforms)

def collate_fn(examples):
    pixel_values = torch.stack([example["pixel_values"] for example in examples])
    labels = torch.tensor([example["label"] for example in examples])
    return {"pixel_values": pixel_values, "labels": labels}

model = ViTForImageClassification.from_pretrained(
    model_name_or_path,
    num_labels=num_labels, # 3으로 자동 설정됨
    id2label=id2label,
    label2id=label2id
)
```


트레이닝 파이션 스크립트 해석

트레이너 설정 및 훈련 시작

```
from transformers import TrainingArguments, Trainer
from sklearn.metrics import accuracy_score

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return dict(accuracy=accuracy_score(predictions, labels))

args = TrainingArguments( f"custom-vit-finetune",
    save_strategy="epoch",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=10,
    per_device_eval_batch_size=4,
    num_train_epochs=5,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    logging_dir='logs_custom',
    remove_unused_columns=False, )
```

```
trainer = Trainer(
    model,
    args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    data_collator=collate_fn,
    compute_metrics=compute_metrics, tokenizer=processor,
)

print("\n--- 커스텀 데이터셋 파인 튜닝 시작 ---")
trainer.train()

print("\n--- 최종 모델 성능 평가 시작 ---")
```

트레이닝 파이션 스크립트 해석

결과 출력 및 모델 저장

```
results = trainer.evaluate(eval_dataset=val_ds)
print(results)
final_accuracy = results['eval_accuracy']
print(f"\n최종 모델의 검증 정확도 (Accuracy): {final_accuracy:.4f}")
output_dir = "final_custom_vit_model"
trainer.model.save_pretrained(output_dir)
processor.save_pretrained(output_dir)
print(f"\n최종 모델이 {output_dir}에 저장되었습니다.")
```

평가 결과

검증 데이터셋에 대한 최종 정확도 출력

모델 저장

final_custom_vit_model 디렉토리에 모델과 프로세서 저장